



CS 6220 Data Mining — Assignment 2

Due: Sep, 2026(100 points)

Yuhong Deng
[GIT REPOSITORY](#) (if applicable)

Q2.1 - Loss Function Gradient

The binary cross-entropy loss is

$$\mathcal{L}(\hat{y}, y) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right].$$

Taking the derivative with respect to $\hat{y}^{(i)}$,

$$\frac{\partial \mathcal{L}}{\partial \hat{y}^{(i)}} = -\frac{1}{N} \left[y^{(i)} \frac{\partial}{\partial \hat{y}^{(i)}} \log \hat{y}^{(i)} + (1 - y^{(i)}) \frac{\partial}{\partial \hat{y}^{(i)}} \log (1 - \hat{y}^{(i)}) \right].$$

Using

$$\frac{d}{dx} \log x = \frac{1}{x}, \quad \frac{d}{dx} \log(1 - x) = -\frac{1}{1 - x},$$

we obtain

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \hat{y}^{(i)}} &= -\frac{1}{N} \left[\frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} \right] \\ &= \frac{1}{N} \left[\frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} - \frac{y^{(i)}}{\hat{y}^{(i)}} \right] \\ &= \frac{1}{N} \frac{\hat{y}^{(i)}(1 - y^{(i)}) - y^{(i)}(1 - \hat{y}^{(i)})}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \\ &= \frac{1}{N} \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})}. \end{aligned}$$

Therefore,

$$\frac{\partial \mathcal{L}}{\partial \hat{y}^{(i)}} = \frac{1}{N} \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)} (1 - \hat{y}^{(i)})}$$

Q2.2 - Gradient Derivation for Sigmoid

The sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}} = (1 + e^{-z})^{-1}.$$

Taking the derivative with respect to z ,

$$\frac{\partial \sigma(z)}{\partial z} = - (1 + e^{-z})^{-2} \frac{\partial}{\partial z} (1 + e^{-z}).$$

Since

$$\frac{\partial}{\partial z} e^{-z} = -e^{-z},$$

we obtain

$$\begin{aligned} \frac{\partial \sigma(z)}{\partial z} &= - (1 + e^{-z})^{-2} (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}}. \end{aligned}$$

Also,

$$\begin{aligned} \sigma(z) &= \frac{1}{1 + e^{-z}}, \\ 1 - \sigma(z) &= 1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}. \end{aligned}$$

Therefore,

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z) (1 - \sigma(z))$$

Q2.3 - Gradient for a Single Layer

For the function

$$\hat{y} = \sigma(Wh + b),$$

let

$$z = Wh + b.$$

Then

$$\hat{y} = \sigma(z).$$

From Q2.2,

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}).$$

Using the chain rule, for W ,

$$\begin{aligned} \frac{\partial \hat{y}}{\partial W} &= \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W} \\ &= \hat{y}(1 - \hat{y})h. \end{aligned}$$

For the input h ,

$$\begin{aligned} \frac{\partial \hat{y}}{\partial h} &= \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial h} \\ &= \hat{y}(1 - \hat{y})W. \end{aligned}$$

For the bias b ,

$$\begin{aligned} \frac{\partial \hat{y}}{\partial b} &= \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b} \\ &= \hat{y}(1 - \hat{y}). \end{aligned}$$

Therefore,

$$\boxed{\frac{\partial \hat{y}}{\partial W} = \hat{y}(1 - \hat{y})h}$$

$$\boxed{\frac{\partial \hat{y}}{\partial h} = \hat{y}(1 - \hat{y})W}$$

$$\boxed{\frac{\partial \hat{y}}{\partial b} = \hat{y}(1 - \hat{y})}$$

Q2.4 - Gradient Derivation for All Parameters

For the two-layer neural network,

$$\begin{aligned} z_1 &= W_1 X + b_1, \\ h &= \sigma(z_1), \\ z_2 &= W_2 h + b_2, \\ \hat{y} &= \sigma(z_2). \end{aligned}$$

From Q2.1 and Q2.2,

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \hat{y}} &= \frac{1}{N} \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}, \\ \frac{\partial \hat{y}}{\partial z_2} &= \hat{y}(1 - \hat{y}).\end{aligned}$$

Therefore,

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial z_2} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \\ &= \frac{1}{N} \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \hat{y}(1 - \hat{y}) \\ &= \frac{1}{N} (\hat{y} - y).\end{aligned}$$

Let

$$\delta_2 = \frac{\partial \mathcal{L}}{\partial z_2} = \frac{1}{N} (\hat{y} - y).$$

For W_2 ,

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_2} &= \delta_2 h^T \\ &= \frac{1}{N} (\hat{y} - y) h^T.\end{aligned}$$

For b_2 ,

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial b_2} &= \sum_{i=1}^N \delta_2^{(i)} \\ &= \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)}).\end{aligned}$$

Next, propagate the gradient back to the first layer.

From Q2.2,

$$\frac{\partial h}{\partial z_1} = h(1 - h).$$

Using the chain rule,

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial z_1} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial h} \odot \frac{\partial h}{\partial z_1} \\ &= \frac{1}{N} \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \hat{y}(1 - \hat{y}) W_2^T \odot h(1 - h) \\ &= \frac{1}{N} [W_2^T (\hat{y} - y)] \odot h(1 - h).\end{aligned}$$

Let

$$\delta_1 = \frac{\partial \mathcal{L}}{\partial z_1} = \frac{1}{N} [W_2^T (\hat{y} - y)] \odot h(1 - h).$$

For W_1 ,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_1} &= \delta_1 X^T \\ &= \frac{1}{N} [W_2^T (\hat{y} - y) \odot h(1 - h)] X^T. \end{aligned}$$

For b_1 ,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial b_1} &= \sum_{i=1}^N \delta_1^{(i)} \\ &= \frac{1}{N} \sum_{i=1}^N [W_2^T (\hat{y} - y) \odot h(1 - h)]^{(i)}. \end{aligned}$$

Therefore, the four parameter gradients are

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_2} = \frac{1}{N} (\hat{y} - y) h^T}$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial b_2} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})}$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_1} = \frac{1}{N} [W_2^T (\hat{y} - y) \odot h(1 - h)] X^T}$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial b_1} = \frac{1}{N} \sum_{i=1}^N [W_2^T (\hat{y} - y) \odot h(1 - h)]^{(i)}}$$

where $\odot$ denotes element-wise multiplication.

Q5 - Optimization Results

The neural network was trained for 100,000 iterations. The training loss decreased steadily during optimization, indicating that the model converged successfully.

As shown in Figure 0.1, the training loss decreased from approximately 0.69 at the beginning of training to approximately 0.03–0.04 near the end of training.

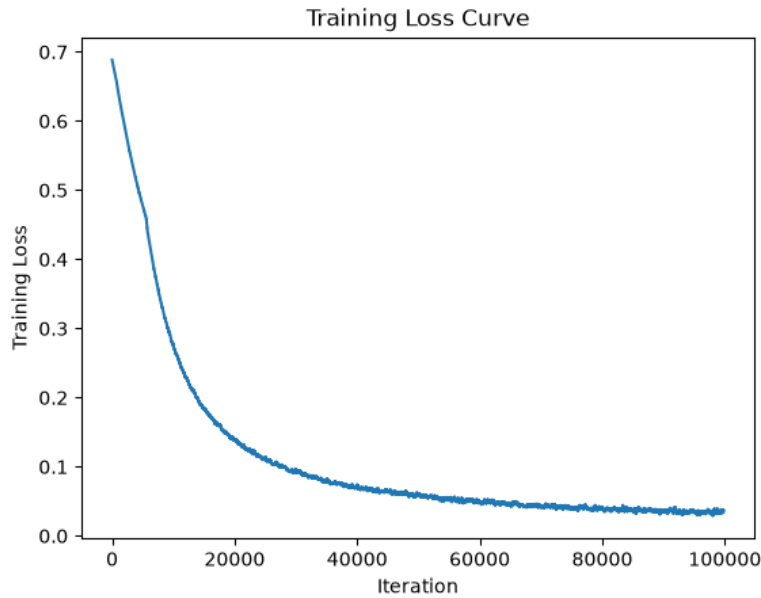


Figure 0.1: Training loss over 100,000 optimization iterations.

Test Set Evaluation

After training, the model was evaluated on the held-out test set. The model achieved the following results:

Test BCE Loss = 0.0321,
Test Accuracy = 99.6%.

The low test loss and high classification accuracy indicate that the trained neural network generalizes well to unseen test data.

Examples from the Test Set

Several examples from the test set were passed through the trained two-layer neural network. The predicted probability represents the probability of positive sentiment. A probability greater than 0.5 is classified as positive sentiment, while a probability less than or equal to 0.5 is classified as negative sentiment.

- Tweet:** Bro:U wan cut hair anot,ur hair long Liao bo Me:since ord liao,take it easy lor treat as save \$ leave it longer :) Bro:LOL SibeI xialan
True label: Positive
Predicted probability: 0.9754
Predicted label: Positive
- Tweet:** @heyclaireee is back! thnx God!!! i'm so happy :)
True label: Positive
Predicted probability: 0.9754
Predicted label: Positive

- **Tweet:** @BBCRadio3 thought it was my ears which were malfunctioning, thank goodness you cleared that one up with an apology :-)
True label: Positive
Predicted probability: 0.9754
Predicted label: Positive
- **Tweet:** @HumayAG ‘Stuck in the centre right with you. Clowns to the right, jokers to the left...’ :) @orgasticpotency @ahmedshaheed @AhmedSaeedGahaa
True label: Positive
Predicted probability: 0.9754
Predicted label: Positive
- **Tweet:** Happy Friday :-) <http://t.co/iymPIlWXYF>
True label: Positive
Predicted probability: 0.9754
Predicted label: Positive

All five examples above were correctly classified as positive sentiment.